

GROUP MEMBER	
MUDASSIR GHAZI	41015

PROJECT:
BLOOD BANK MANAGEMENT SYSTEM

INSTRUCTOR:
MAAM SANA YOUSAF







Table of Contents

1. Introduction 1
2. Purpose 1
3. Entity Relationship Diagram (ERD) 2
4. Normalization (UNF → 1NF → 2NF → 3NF → BCNF) 3
4.1 Unnormalized Form (UNF) 3
4.2 First Normal Form (1NF) 3
4.3 Second Normal Form (2NF) 6
4.4 Third Normal Form (3NF) 7
4.5 Boyce–Codd Normal Form (BCNF) 8
4.6 Final Normalized Schema (BCNF) 9
5. SQL Developer 10
5.1 Table Creation (DDL) 10
5.3 Retrieval Queries (Simple, Join, Subquery, Aggregate) 14
5.4 View, Indexes & Sequence 16
5.5 PL/SQL Objects Procedure, Function, Trigger, Cursor 17
6. Frontend Java Swing Application 19
6.1 Login & Registration (AuthFrame) 19
6.2 Main Dashboard (BBMSApp) 20

1. Introduction

LifeLine is a desktop-based Blood Bank Management System (BBMS) developed to efficiently manage donor registration, blood testing, inventory, and patient blood requests. It replaces manual record keeping with a centralized Oracle relational database, improving data accuracy, consistency, and accessibility.

The system consists of an ER Diagram, Oracle SQL database, and Java Swing frontend for Admins, Donors, and Patients. The database implementation includes DDL, DML, queries, views, indexes, sequences, and PL/SQL objects.

LifeLine follows a layered architecture with a Presentation Layer, Data Access Layer (DAO), and Persistence Layer (Oracle Database) connected through JDBC. This structure makes the system organized, maintainable, and easy to extend.

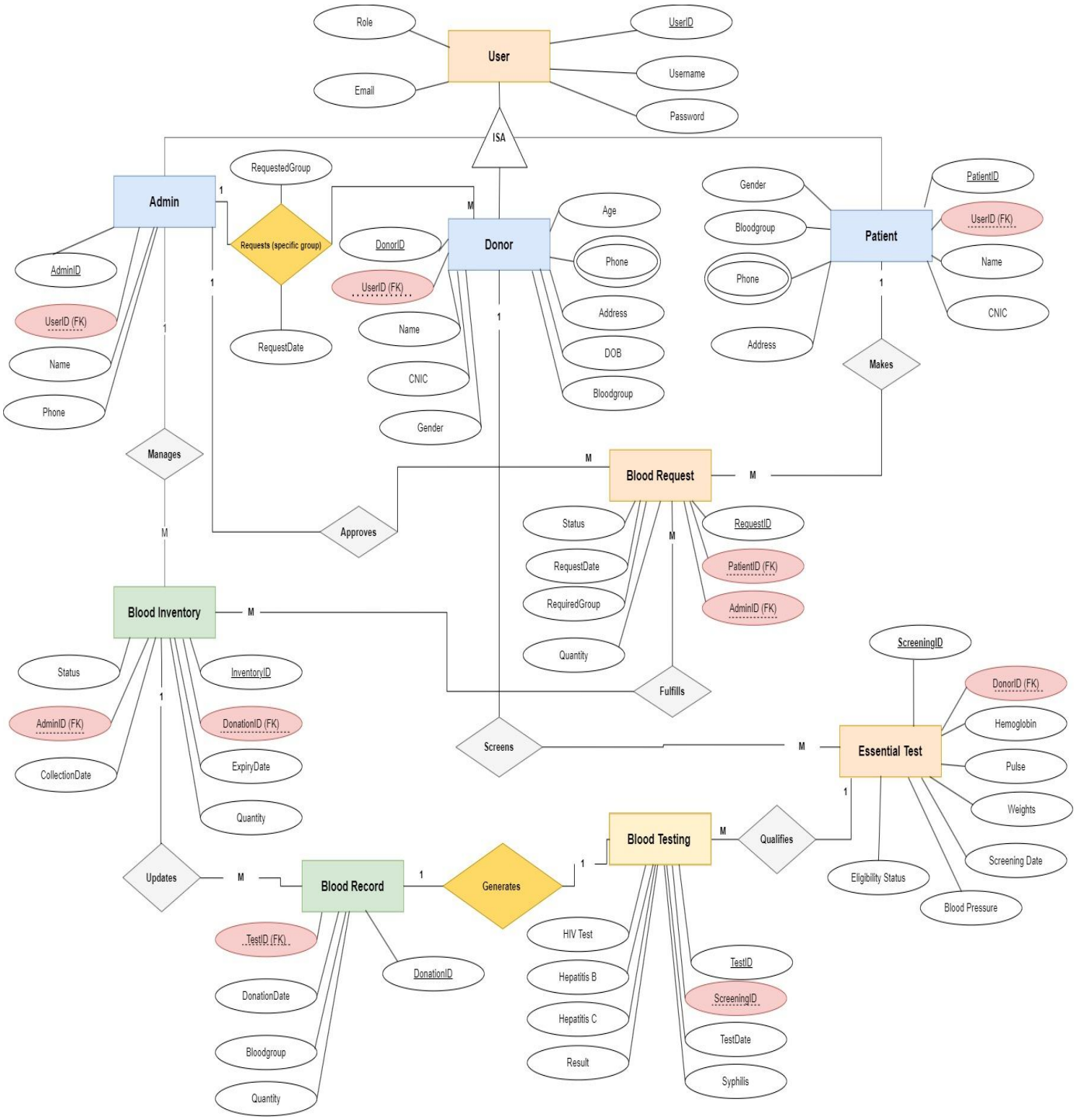
2. Purpose

The purpose of this project is to design and implement a reliable, well-structured database system that supports the day-to-day operations of a blood bank while eliminating the data redundancy and integrity issues found in unnormalized, spreadsheet-style record keeping. Specifically, the system aims to:

- Maintain a single source of truth for Users, Admins, Donors, and Patients through a proper ISA (generalization) structure.
- Track the full donation pipeline — from donor screening (Essential Test), to blood testing, to the blood record, and finally to inventory stock — with clear traceability at every stage.
- Allow Patients to raise blood requests and Admins to review, approve, or reject them against available inventory.
- Enforce data integrity through primary keys, foreign keys, and check constraints (e.g. valid blood groups, donor age limits, request status values).
- Provide a normalized (BCNF-compliant) database schema that is free of repeating groups, partial dependencies, and transitive dependencies.
- Provide a working, user-friendly desktop interface so non-technical staff (Admins, Donors, Patients) can use the system without writing any SQL themselves.

3. Entity Relationship Diagram (ERD)

The ER diagram below models the entities of the system User, Admin, Donor, Patient, Blood Request, Essential Test, Blood Testing, Blood Record, and Blood Inventory along with their relationships (Manages, Approves, Screens, Qualifies, Generates, Updates, Makes, Fulfills, and the Requests-specific-group weak relationship between Admin and Donor). The User entity is specialized into Admin, Donor, and Patient through an ISA relationship.



4. Normalization (UNF → 1NF → 2NF → 3NF → BCNF)

Before building the database, the system's data was first expressed as a single unnormalized relation (UNF) and then progressively decomposed into normal forms. Two one-to-many relationships in the ER diagram give rise to nested repeating groups: a single Donor can undergo multiple Essential Tests (which chains further into Blood Testing → Blood Record → Blood Inventory), and a single Patient can raise multiple Blood Requests.

4.1 Unnormalized Form (UNF)

All attributes of the system were first bundled into one relation, BBMS, with the repeating groups nested inside it exactly as they occur in the real-world process:

```
BBMS (
  UserID, Role, Username, Password, Email,
  AdminID, AdminName, AdminPhone,
  DonorID, DonorName, DonorCNIC, DonorGender, DonorAge, DonorPhone, DonorAddress,
  DonorDOB, DonorBloodgroup,
    ( ScreeningID, Hemoglobin, Pulse, Weight, ScreeningDate, EligibilityStatus, BloodPressure,
      ( TestID, TestDate, HIVTest, HepatitisB, HepatitisC, Syphilis, Result,
        ( DonationID, DonationDate, DonationBloodgroup, DonationQuantity,
          ( InventoryID, CollectionDate, ExpiryDate, InventoryQuantity, InventoryStatus )*
        )*
      )*
    )*,
  PatientID, PatientName, PatientCNIC, PatientGender, PatientBloodgroup, PatientPhone,
  PatientAddress,
    ( RequestID, RequestDate, RequiredGroup, RequestQuantity, RequestStatus )*
)
```

4.2 First Normal Form (1NF)

1NF requires that every repeating group be removed into its own relation, with the primary key of the enclosing relation copied down as a foreign key. This was done in four rounds, working from the outermost repeating groups inward.

Round 1 Outermost repeating groups

Two repeating groups sit directly at the top level of BBMS: the Essential Test block (with everything else nested inside it) and the Request block. Both are extracted first.

Extracted Relation	Attributes	Primary Key	FK copied down	Source repeating group
ESSENTIALTEST-1	DonorID, ScreeningID, Hemoglobin, Pulse, Weight, ScreeningDate, EligibilityStatus, BloodPressure	ScreeningID	DonorID	Essential Test block (Donor is Screened)
REQUEST-1	PatientID, RequestID, RequestDate, RequiredGroup, RequestQuantity, RequestStatus	RequestID	PatientID	Request block (Patient Makes a request)

Round 2 nested inside ESSENTIALTEST-1

A Blood Testing group was still nested inside ESSENTIALTEST-1 and was extracted next, carrying ScreeningID down as a foreign key.

Extracted Relation	Attributes	Primary Key	FK copied down	Source repeating group
BLOODTESTING-1	ScreeningID, TestID, TestDate, HIVTest, HepatitisB, HepatitisC, Syphilis, Result	TestID	ScreeningID	Blood Testing block, nested in Essential Test

Round 3 nested inside BLOODTESTING-1

A Blood Record group was still nested inside BLOODTESTING-1 and was extracted next, carrying TestID down as a foreign key.

Extracted Relation	Attributes	Primary Key	FK copied down	Source repeating group
BLOODRECORD-1	TestID, DonationID, DonationDate, DonationBloodgroup, DonationQuantity	DonationID	TestID	Blood Record block, nested in Blood Testing

Round 4 last nested group, inside BLOODRECORD-1

The final nested group, Blood Inventory, was extracted last. AdminID was added here as well, since the ERD shows Admin manages/updates the inventory.

Extracted Relation	Attributes	Primary Key	FK copied down	Source repeating group
BLOODINVENTORY-1	DonationID, AdminID, InventoryID, CollectionDate, ExpiryDate, InventoryQuantity, InventoryStatus	InventoryID	DonationID, AdminID	Blood Inventory block, nested in Blood Record

What remains of the original relation

After all nested repeating groups were extracted, the remaining base attributes were also separated — User, Admin, Donor, and Patient are distinct entities that were only bundled together by the ISA (generalization) relationship, so each is split into its own relation with UserID retained as a foreign key.

Relation	Attributes	Primary Key	Foreign Key
USER-1	UserID, Role, Username, Password, Email	UserID	—

Relation	Attributes	Primary Key	Foreign Key
ADMIN-1	AdminID, UserID, AdminName, AdminPhone	AdminID	UserID
DONOR-1	DonorID, UserID, DonorName, DonorCNIC, DonorGender, DonorAge, DonorPhone, DonorAddress, DonorDOB, DonorBloodgroup	DonorID	UserID
PATIENT-1	PatientID, UserID, PatientName, PatientCNIC, PatientGender, PatientBloodgroup, PatientPhone, PatientAddress	PatientID	UserID

In addition, the Admin–Donor relationship "Requests a specific blood group" carries its own attributes (RequestedGroup, RequestDate) and has no independent existence outside that pairing, so it is represented as a separate relation with a composite primary key:

Relation	Attributes	Primary Key	Foreign Keys
REQUESTGROUP-1	AdminID, DonorID, RequestedGroup, RequestDate	(AdminID, DonorID) — composite	AdminID, DonorID

4.3 Second Normal Form (2NF)

A relation is in 2NF if it is already in 1NF and every non-key attribute is fully dependent on the whole primary key, not on only part of it. This rule only matters for relations with a composite primary key, since a single-attribute key cannot produce a partial dependency.

Nine of the ten 1NF relations (USER, ADMIN, DONOR, PATIENT, REQUEST, ESSENTIAL TEST, BLOOD TESTING, BLOOD RECORD, BLOOD INVENTORY) each have a single-attribute primary key, so they satisfy 2NF automatically. The only relation with a composite key is REQUEST GROUP (AdminID, DonorID):

Composite Key	Dependent Attribute	Depends on full key or part of it?
(AdminID, DonorID)	RequestedGroup	Full key — only meaningful for a specific Admin requesting from a specific Donor
(AdminID, DonorID)	RequestDate	Full key — same reasoning applies

Conclusion: no partial dependency exists in any relation of the BBMS schema. All ten relations satisfy 2NF.

4.4 Third Normal Form (3NF)

A relation is in 3NF if it is already in 2NF and no non-key attribute is transitively dependent on the primary key — i.e. no non-key attribute determines another non-key attribute. Each relation was checked individually:

Relation	**Functional Dependency**	**Transitive Dependency?**
USER	UserID → Role, Username, Password, Email	Not present
ADMIN	AdminID → UserID, AdminName, AdminPhone	Not present
DONOR	DonorID → UserID, DonorName, DonorCNIC, DonorGender, DonorAge, DonorPhone, DonorAddress, DonorDOB, DonorBloodgroup	Not present
PATIENT	PatientID → UserID, PatientName, PatientCNIC, PatientGender, PatientBloodgroup, PatientPhone, PatientAddress	Not present
REQUEST	RequestID → PatientID, AdminID, RequestDate, RequiredGroup, RequestQuantity, RequestStatus	Not present
ESSENTIAL TEST	ScreeningID → DonorID, Hemoglobin, Pulse, Weight, ScreeningDate, EligibilityStatus, BloodPressure	Not present
BLOOD TESTING	TestID → ScreeningID, TestDate, HIVTest, HepatitisB, HepatitisC, Syphilis, Result	Not present*
BLOOD RECORD	DonationID → TestID, DonationDate, DonationBloodgroup, DonationQuantity	Not present
BLOOD INVENTORY	InventoryID → AdminID, DonationID, CollectionDate, ExpiryDate, InventoryQuantity, InventoryStatus	Not present
REQUEST GROUP	(AdminID, DonorID) → RequestedGroup, RequestDate	Not present

*Note: In practice, Result is influenced by the outcomes of HIVTest, HepatitisB, HepatitisC, and Syphilis. This is treated as a business rule/derived value rather than a strict functional dependency between non-key attributes, since Result is still recorded independently. All ten relations satisfy 3NF.

4.5 Boyce–Codd Normal Form (BCNF)

A relation is in BCNF if and only if every determinant is a candidate key — a stricter requirement than 3NF. Every determinant in every relation was checked against the candidate keys of that relation:

Relation	Determinant	Candidate Key(s)	Is Determinant a Candidate Key?
USER	UserID	UserID, Username	Yes
ADMIN	AdminID	AdminID, UserID	Yes
DONOR	DonorID	DonorID, DonorCNIC	Yes
PATIENT	PatientID	PatientID, PatientCNIC	Yes
REQUEST	RequestID	RequestID	Yes
ESSENTIAL TEST	ScreeningID	ScreeningID	Yes
BLOOD TESTING	TestID	TestID, ScreeningID	Yes
BLOOD RECORD	DonationID	DonationID, TestID	Yes
BLOOD INVENTORY	InventoryID	InventoryID, DonationID	Yes
REQUEST GROUP	(AdminID, DonorID)	(AdminID, DonorID)	Yes

Note: ScreeningID in BLOOD TESTING is a foreign key only, not an alternate candidate key, since the "Qualifies" relationship between Essential Test and Blood Testing is one-to-many (1 Essential Test : M Blood Testing) as shown in the ER diagram. Every determinant in every relation is a candidate key, so the entire BBMS schema is in BCNF.

4.6 Final Normalized Schema (BCNF)

The table below is the final, fully normalized relational schema that was implemented in Oracle Database — it is free of repeating groups, partial dependencies, and transitive dependencies.

No.	Relation	Attributes	Primary Key	Foreign Key(s)
1	USER	UserID, Role, Username, Password, Email	UserID	—
2	ADMIN	AdminID, UserID, AdminName, AdminPhone	AdminID	UserID → USER
3	DONOR	DonorID, UserID, DonorName, DonorCNIC, DonorGender, DonorAge, DonorPhone, DonorAddress, DonorDOB, DonorBloodgroup	DonorID	UserID → USER
4	PATIENT	PatientID, UserID, PatientName, PatientCNIC, PatientGender, PatientBloodgroup, PatientPhone, PatientAddress	PatientID	UserID → USER
5	REQUEST	RequestID, PatientID, AdminID, RequestDate, RequiredGroup, RequestQuantity, RequestStatus	RequestID	PatientID → PATIENT; AdminID → ADMIN
6	ESSENTIAL TEST	ScreeningID, DonorID, Hemoglobin, Pulse, Weight, ScreeningDate, EligibilityStatus, BloodPressure	ScreeningID	DonorID → DONOR
7	BLOOD TESTING	TestID, ScreeningID, TestDate, HIVTest, HepatitisB, HepatitisC, Syphilis, Result	TestID	ScreeningID → ESSENTIAL TEST
8	BLOOD RECORD	DonationID, TestID, DonationDate, DonationBloodgroup, DonationQuantity	DonationID	TestID → BLOOD TESTING
9	BLOOD INVENTORY	InventoryID, AdminID, DonationID, CollectionDate, ExpiryDate, InventoryQuantity, InventoryStatus	InventoryID	AdminID → ADMIN; DonationID → BLOOD RECORD

No.	Relation	Attributes	Primary Key	Foreign Key(s)
10	REQUEST GROUP	AdminID, DonorID, RequestedGroup, RequestDate	(AdminID, DonorID)	AdminID → ADMIN; DonorID → DONOR

5. SQL Developer

Since the course focus of this project is the database (backend), this section reproduces the entire SQL script exactly as implemented and tested in Oracle SQL Developer — every CREATE TABLE, every INSERT batch, every retrieval query, the view/index/sequence, and all four PL/SQL objects — each followed by a short explanation of what it does and why it is written that way. All of this is combined into one file (ALL_QUERIES_COMBINED.sql) that can be run top to bottom (F5 in SQL Developer) to rebuild the whole database from scratch.

5.1 Table Creation (DDL)

The script first drops any existing tables (in reverse dependency order, using CASCADE CONSTRAINTS) and then rebuilds all nine tables of the BCNF schema, with every primary key, foreign key, and CHECK constraint defined inline.

```
DROP TABLE BLOOD_INVENTORY CASCADE CONSTRAINTS;
DROP TABLE BLOOD_RECORDS CASCADE CONSTRAINTS;
DROP TABLE BLOOD_TESTING CASCADE CONSTRAINTS;
DROP TABLE ESSENTIAL_TESTS CASCADE CONSTRAINTS;
DROP TABLE BLOOD_REQUESTS CASCADE CONSTRAINTS;
DROP TABLE PATIENTS CASCADE CONSTRAINTS;
DROP TABLE DONORS CASCADE CONSTRAINTS;
DROP TABLE ADMINS CASCADE CONSTRAINTS;
DROP TABLE USERS CASCADE CONSTRAINTS;
```

Tables are dropped child-first so that foreign-key dependencies don't block the DROP; CASCADE CONSTRAINTS removes any dependent constraints automatically.

1. USERS base entity (Admin / Donor / Patient all ISA a User)

```
CREATE TABLE USERS (
   UserID      VARCHAR2(10)  PRIMARY KEY,
   Username    VARCHAR2(50)  NOT NULL UNIQUE,
   Email       VARCHAR2(100) NOT NULL UNIQUE,
   Password    VARCHAR2(50)  NOT NULL,
```

```
  Role       VARCHAR2(10)  NOT NULL,
  CONSTRAINT chk_user_role CHECK (Role IN ('Admin','Donor','Patient'))
);
```

Holds the login credentials shared by every account type. The CHECK constraint restricts Role to exactly the three values the ISA hierarchy supports, and Username/Email are UNIQUE so no two accounts can collide.

2. ADMINS

```
CREATE TABLE ADMINS (
  AdminID    VARCHAR2(10) PRIMARY KEY,
  UserID     VARCHAR2(10) NOT NULL UNIQUE,
  Name       VARCHAR2(50) NOT NULL,
  Phone      VARCHAR2(15),
  CONSTRAINT fk_admin_user FOREIGN KEY (UserID) REFERENCES USERS(UserID)
);
```

One row per Admin. UserID is UNIQUE as well as a foreign key, which enforces the 1-to-1 ISA relationship between USERS and ADMINS.

3. DONORS

```
CREATE TABLE DONORS (
  DonorID    VARCHAR2(10) PRIMARY KEY,
  UserID     VARCHAR2(10) NOT NULL UNIQUE,
  Name       VARCHAR2(50) NOT NULL,
  CNIC       VARCHAR2(15) NOT NULL UNIQUE,
  Gender     VARCHAR2(10),
  Age        NUMBER(3),
  Phone      VARCHAR2(15),
  Address    VARCHAR2(100),
  DOB        DATE,
  Bloodgroup  VARCHAR2(3),
  CONSTRAINT fk_donor_user FOREIGN KEY (UserID) REFERENCES USERS(UserID),
  CONSTRAINT chk_donor_gender CHECK (Gender IN ('Male','Female','Other')),
  CONSTRAINT chk_donor_age CHECK (Age BETWEEN 18 AND 65),
  CONSTRAINT chk_donor_group CHECK (Bloodgroup IN ('A+','A-','B+','B-','AB+','AB-','O+','O-'))
);
```

Donor profile data. CNIC is UNIQUE to prevent duplicate registrations, the Age CHECK enforces the legal/medical donation age window (18–65), and Bloodgroup is restricted to the eight valid blood types.

4. PATIENTS

```
CREATE TABLE PATIENTS (
  PatientID  VARCHAR2(10) PRIMARY KEY,
  UserID     VARCHAR2(10) NOT NULL UNIQUE,
```

```
  Name       VARCHAR2(50) NOT NULL,
  CNIC       VARCHAR2(15) NOT NULL UNIQUE,
  Gender     VARCHAR2(10),
  Bloodgroup VARCHAR2(3),
  Phone      VARCHAR2(15),
  Address    VARCHAR2(100),
  CONSTRAINT fk_patient_user FOREIGN KEY (UserID) REFERENCES USERS(UserID),
  CONSTRAINT chk_patient_gender CHECK (Gender IN ('Male','Female','Other')),
  CONSTRAINT chk_patient_group CHECK (Bloodgroup IN ('A+','A-','B+','B-','AB+','AB-','O+','O-'))
);
```

Mirrors the DONORS table structure for Patients, with the same CNIC-uniqueness and blood-group validation rules.

5. BLOOD_REQUESTS

Patient Makes a request; Admin Approves it

```
-- NOTE: the ERD's "Requests (specific group)" weak entity is merged in here
-- as RequiredGroup + RequestDate, since it has no independent existence.
CREATE TABLE BLOOD_REQUESTS (
  RequestID    VARCHAR2(10) PRIMARY KEY,
  PatientID    VARCHAR2(10) NOT NULL,
  AdminID      VARCHAR2(10),
  Status       VARCHAR2(10) DEFAULT 'Pending',
  RequestDate  DATE DEFAULT SYSDATE,
  RequiredGroup VARCHAR2(3),
  Quantity     NUMBER(3),
  CONSTRAINT fk_request_patient FOREIGN KEY (PatientID) REFERENCES PATIENTS(PatientID),
  CONSTRAINT fk_request_admin   FOREIGN KEY (AdminID)   REFERENCES ADMINS(AdminID),
  CONSTRAINT chk_request_status CHECK (Status IN ('Pending','Approved','Rejected','Fulfilled')),
  CONSTRAINT chk_request_group  CHECK (RequiredGroup IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')),
  CONSTRAINT chk_request_qty    CHECK (Quantity > 0)
);
```

AdminID is nullable because a request starts life as Pending with no admin assigned yet, and only gets an AdminID once someone approves or rejects it. Status defaults to 'Pending' and RequestDate defaults to the current system date.

6. ESSENTIAL_TESTS Donor is Screened

```
-- Quantity is an addition beyond the original ERD: units of blood actually
-- drawn, entered once here when the donor is Eligible, then reused
-- automatically by Blood Record and Blood Inventory later in the pipeline.
CREATE TABLE ESSENTIAL_TESTS (
  ScreeningID    VARCHAR2(10) PRIMARY KEY,
  DonorID        VARCHAR2(10) NOT NULL,
```

```
    Hemoglobin       NUMBER(4,1),
    Pulse            NUMBER(3),
    Weights          NUMBER(5,1),
    ScreeningDate    DATE DEFAULT SYSDATE,
    BloodPressure    VARCHAR2(10),
    EligibilityStatus VARCHAR2(10),
    Quantity         NUMBER(3) DEFAULT 0,
    CONSTRAINT fk_screen_donor FOREIGN KEY (DonorID) REFERENCES DONORS(DonorID),
    CONSTRAINT chk_screen_elig CHECK (EligibilityStatus IN ('Eligible','Ineligible','Pending'))
);
```

Records the pre-donation health screening (hemoglobin, pulse, weight, blood pressure) for a Donor and the resulting eligibility decision, which gates whether the donor's blood proceeds to Blood Testing.

7. BLOOD_TESTING — only Qualified/Eligible screenings reach here

```
-- Result allows 'Pending' so a row can sit here the instant a screening
-- turns Eligible, before the lab results are actually typed in.
CREATE TABLE BLOOD_TESTING (
    TestID       VARCHAR2(10) PRIMARY KEY,
    ScreeningID  VARCHAR2(10) NOT NULL UNIQUE,
    TestDate     DATE DEFAULT SYSDATE,
    HIVTest      VARCHAR2(10),
    HepatitisB   VARCHAR2(10),
    HepatitisC   VARCHAR2(10),
    Syphilis     VARCHAR2(10),
    Result       VARCHAR2(10),
    CONSTRAINT fk_test_screening FOREIGN KEY (ScreeningID) REFERENCES ESSENTIAL_TESTS(ScreeningID),
    CONSTRAINT chk_test_result CHECK (Result IN ('Pass','Fail','Pending'))
);
```

Each eligible screening produces exactly one blood test (ScreeningID is UNIQUE), which records the four infectious-disease screens and an overall Pass/Fail/Pending result.

8. BLOOD_RECORDS

Generated from a Pass result

```
-- Quantity allows 0 so a record can be auto-created the instant a test
-- Passes, with the real number filled in moments later from Essential Test.
CREATE TABLE BLOOD_RECORDS (
    DonationID    VARCHAR2(10) PRIMARY KEY,
    TestID        VARCHAR2(10) NOT NULL,
    DonationDate  DATE DEFAULT SYSDATE,
    Bloodgroup    VARCHAR2(3),
    Quantity      NUMBER(3),
    CONSTRAINT fk_record_test FOREIGN KEY (TestID) REFERENCES BLOOD_TESTING(TestID),
```

```
  CONSTRAINT chk_record_qty CHECK (Quantity >= 0)
);
```

A donation record is only ever generated from a Blood Testing row whose Result is 'Pass' — this is the 'Generates' relationship from the ER diagram.

9. BLOOD_INVENTORY Updated by new records; Managed by Admin; Fulfils requests

```
-- Status allows 'Pending' for the same auto-queue reason as above.
CREATE TABLE BLOOD_INVENTORY (
  InventoryID    VARCHAR2(10) PRIMARY KEY,
  DonationID     VARCHAR2(10) NOT NULL UNIQUE,
  AdminID        VARCHAR2(10) NOT NULL,
  Status         VARCHAR2(10) DEFAULT 'Available',
  CollectionDate DATE,
  ExpiryDate     DATE,
  Quantity       NUMBER(3),
  CONSTRAINT fk_inv_record FOREIGN KEY (DonationID) REFERENCES BLOOD_RECORDS(DonationID),
  CONSTRAINT fk_inv_admin  FOREIGN KEY (AdminID)    REFERENCES ADMINS(AdminID),
  CONSTRAINT chk_inv_status CHECK (Status IN ('Available','Reserved','Expired','Pending')),
  CONSTRAINT chk_inv_qty    CHECK (Quantity >= 0)
);


COMMIT;
```

The final stage of the pipeline: one inventory row per donation record, tracking stock status (Available/Reserved/Expired/Pending), collection date, and expiry date. The Admin who logged the stock is recorded via AdminID.

5.3 Retrieval Queries (Simple, Join, Subquery, Aggregate)

Seven SELECT queries were written to demonstrate the different query categories required by the course: simple filtering, joins, subqueries, and aggregation.

Query 1 Simple: all O+ donors

```
SELECT DonorID, Name, Bloodgroup, Phone
FROM DONORS
WHERE Bloodgroup = 'O+';
```

A straightforward single-table filter that returns every donor whose blood group is O+, the universal donor type — useful when an O+ unit is urgently needed.

Query 2 Simple: all pending blood requests

```
SELECT RequestID, RequiredGroup, Quantity, RequestDate
FROM BLOOD_REQUESTS
WHERE Status = 'Pending';
```

Another single-table filter, this time on BLOOD_REQUESTS, giving Admin staff a quick worklist of requests still awaiting a decision.

Query 3 Join: patient name + request status + approving admin

```
SELECT br.RequestID, p.Name AS PatientName, br.RequiredGroup,
    br.Status, a.Name AS ApprovedBy
FROM BLOOD_REQUESTS br
JOIN PATIENTS p ON br.PatientID = p.PatientID
LEFT JOIN ADMINS a ON br.AdminID = a.AdminID;
```

An inner JOIN brings in the patient's name, while a LEFT JOIN to ADMINS is used (instead of an inner join) so that Pending requests — which have no AdminID yet — still appear in the result, just with a NULL ApprovedBy.

Query 4 Multi-table join: full donation pipeline, donor to inventory

```
SELECT d.Name AS DonorName, et.ScreeningID, et.EligibilityStatus,
    bt.TestID, bt.Result, brc.DonationID, bi.InventoryID, bi.Status
FROM DONORS d
JOIN ESSENTIAL_TESTS et ON d.DonorID = et.DonorID
JOIN BLOOD_TESTING bt   ON et.ScreeningID = bt.ScreeningID
JOIN BLOOD_RECORDS brc  ON bt.TestID = brc.TestID
JOIN BLOOD_INVENTORY bi ON brc.DonationID = bi.DonationID;
```

Chains four INNER JOINs across the entire donation pipeline (Donor → Essential Test → Blood Testing → Blood Record → Blood Inventory) to trace a single unit of blood from the donor who gave it all the way to its current stock status.

Query 5 Subquery (NOT IN): donors who have never been screened

```
SELECT Name, Bloodgroup
FROM DONORS
WHERE DonorID NOT IN (SELECT DonorID FROM ESSENTIAL_TESTS);
```

A NOT IN subquery finds donors with zero matching rows in ESSENTIAL_TESTS — i.e. donors who registered but have not yet come in for a screening appointment.

Query 6 Subquery (nested): patients whose requested quantity is above average

```
SELECT Name
FROM PATIENTS
WHERE PatientID IN (
```

```
    SELECT PatientID FROM BLOOD_REQUESTS
    WHERE Quantity > (SELECT AVG(Quantity) FROM BLOOD_REQUESTS)
);
```

A doubly-nested subquery: the innermost query computes the average requested Quantity across all requests, the middle query finds which PatientIDs requested more than that average, and the outer query resolves those IDs back to patient names.

Query 7 Aggregate + GROUP BY: available stock per blood group

```
SELECT brc.Bloodgroup, SUM(brc.Quantity) AS TotalUnits
FROM BLOOD_RECORDS brc
JOIN BLOOD_INVENTORY bi ON brc.DonationID = bi.DonationID
WHERE bi.Status = 'Available'
GROUP BY brc.Bloodgroup
ORDER BY TotalUnits DESC;
```

Joins BLOOD_RECORDS to BLOOD_INVENTORY, keeps only rows currently marked 'Available', groups by blood group, and SUMs the Quantity to give a live stock report — the single most operationally useful query in the system, since it tells staff exactly how many units of each blood type are on hand.

5.4 View, Indexes & Sequence

View VW_DONATION_PIPELINE

```
CREATE OR REPLACE VIEW VW_DONATION_PIPELINE AS
SELECT d.DonorID, d.Name AS DonorName, et.ScreeningID, et.EligibilityStatus,
      bt.TestID, bt.Result, brc.DonationID, brc.Bloodgroup,
      bi.InventoryID, bi.Status AS StockStatus
FROM DONORS d
JOIN ESSENTIAL_TESTS et ON d.DonorID = et.DonorID
JOIN BLOOD_TESTING bt   ON et.ScreeningID = bt.ScreeningID
JOIN BLOOD_RECORDS brc  ON bt.TestID = brc.TestID
JOIN BLOOD_INVENTORY bi ON brc.DonationID = bi.DonationID;


-- test it:
-- SELECT * FROM VW_DONATION_PIPELINE;
```

Wraps the four-way join from Query 4 into a permanent, reusable view, so the full donation pipeline can be pulled with a single SELECT * FROM VW_DONATION_PIPELINE instead of retyping the join every time.

Indexes

```
CREATE INDEX idx_donor_bloodgroup ON DONORS(Bloodgroup);
CREATE INDEX idx_patient_cnic     ON PATIENTS(CNIC);
```

Indexes were added on the columns most likely to be searched in real use — Bloodgroup on DONORS (to quickly find donors of a given type) and CNIC on PATIENTS (to quickly look up a patient by their national ID).

Sequence seq_request_id

```
CREATE SEQUENCE seq_request_id START WITH 11 INCREMENT BY 1;


-- usage when inserting a new request:
-- INSERT INTO BLOOD_REQUESTS VALUES ('R' || seq_request_id.NEXTVAL, 'P01', NULL, 'Pending', SYSDATE, 'O+', 1);
```

Since the 10 sample requests already use R01–R10, the sequence starts at 11 so that new RequestIDs generated by the application (R11, R12, …) never collide with the seeded sample data.

5.5 PL/SQL Objects Procedure, Function, Trigger, Cursor

Procedure approve_request

```
CREATE OR REPLACE PROCEDURE approve_request (
   p_requestid IN VARCHAR2,
   p_adminid   IN VARCHAR2
) IS
BEGIN
   UPDATE BLOOD_REQUESTS
   SET Status = 'Approved', AdminID = p_adminid
   WHERE RequestID = p_requestid;

   IF SQL%ROWCOUNT = 0 THEN
      RAISE_APPLICATION_ERROR(-20001, 'Request ID not found: ' || p_requestid);
   END IF;

   COMMIT;
END approve_request;
/


-- test it:
-- EXEC approve_request('R03', 'A01');
-- SELECT * FROM BLOOD_REQUESTS WHERE RequestID = 'R03';
```

Encapsulates the 'Admin Approves a request' business action in one call: it updates the Status to 'Approved' and stamps the AdminID who approved it. SQL%ROWCOUNT is checked after the

UPDATE so that passing a RequestID that doesn't exist raises a clear custom error (-20001) instead of silently doing nothing.

Function get_eligibility

```
CREATE OR REPLACE FUNCTION get_eligibility (
   p_donorid IN VARCHAR2
) RETURN VARCHAR2 IS
   v_status VARCHAR2(10);
BEGIN
   SELECT EligibilityStatus INTO v_status
   FROM ESSENTIAL_TESTS
   WHERE DonorID = p_donorid
   AND ScreeningDate = (SELECT MAX(ScreeningDate) FROM ESSENTIAL_TESTS WHERE DonorID = p_donorid);

   RETURN v_status;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'Not Screened';
END get_eligibility;
/

-- test it:
-- SELECT get_eligibility('D01') FROM DUAL;
```

Given a DonorID, returns that donor's EligibilityStatus from their most recent screening (using MAX(ScreeningDate) to pick the latest one, since a donor can be screened more than once). The EXCEPTION block catches donors with no screening rows at all and returns 'Not Screened' instead of raising an error.

Trigger trg_check_inventory

```
CREATE OR REPLACE TRIGGER trg_check_inventory
BEFORE INSERT OR UPDATE ON BLOOD_INVENTORY
FOR EACH ROW
BEGIN
   IF :NEW.ExpiryDate < SYSDATE THEN
      :NEW.Status := 'Expired';
   END IF;
END;
/

-- test it:
-- INSERT INTO BLOOD_INVENTORY VALUES ('INV10','DN01','A01','Available',
--   TO_DATE('2026-01-01','YYYY-MM-DD'), TO_DATE('2026-02-01','YYYY-MM-DD'), 1);
-- SELECT Status FROM BLOOD_INVENTORY WHERE InventoryID = 'INV10';  -> shows 'Expired'
```

A row-level BEFORE INSERT OR UPDATE trigger that inspects the incoming :NEW.ExpiryDate on every insert/update to BLOOD_INVENTORY. If the expiry date is already in the past, it silently overrides :NEW.Status to 'Expired' — this is what automatically turns INV08 (Section 5.2) into expired stock without any application code having to check dates manually.

Cursor c_pending

```
SET SERVEROUTPUT ON;
DECLARE
   CURSOR c_pending IS
      SELECT RequestID, RequiredGroup, Quantity
      FROM BLOOD_REQUESTS
      WHERE Status = 'Pending';
BEGIN
   FOR rec IN c_pending LOOP
      DBMS_OUTPUT.PUT_LINE('Pending Request: ' || rec.RequestID ||
                           ' | Group: ' || rec.RequiredGroup ||
                           ' | Qty: ' || rec.Quantity);
   END LOOP;
END;
/
```

An explicit CURSOR is declared over all Pending requests, then a FOR loop iterates through it row by row and prints a formatted line for each one via DBMS_OUTPUT.PUT_LINE — a classic PL/SQL row-processing pattern, shown here as a standalone anonymous block (SET SERVEROUTPUT ON must be enabled in SQL Developer to see the printed output).

6. Frontend Java Swing Application

The frontend was built in Java using the Swing GUI toolkit and developed in IntelliJ IDEA. It connects to the Oracle database through the JDBC thin driver (DBConnection.java) and follows the DAO (Data Access Object) design pattern — each entity (User, Admin, Donor, Patient, Blood Request, Essential Test, Blood Testing, Blood Record, Blood Inventory) has its own DAO class responsible for all SQL interaction, keeping the UI code free of embedded queries.

6.1 Login & Registration (AuthFrame)

The application opens on an AuthFrame window titled "LifeLine BBMS", which presents a background-image login form and a registration form for new Donors and Patients. Credentials are validated against the USERS table, and the appropriate role (Admin, Donor, or Patient) is resolved before the main application window is launched.

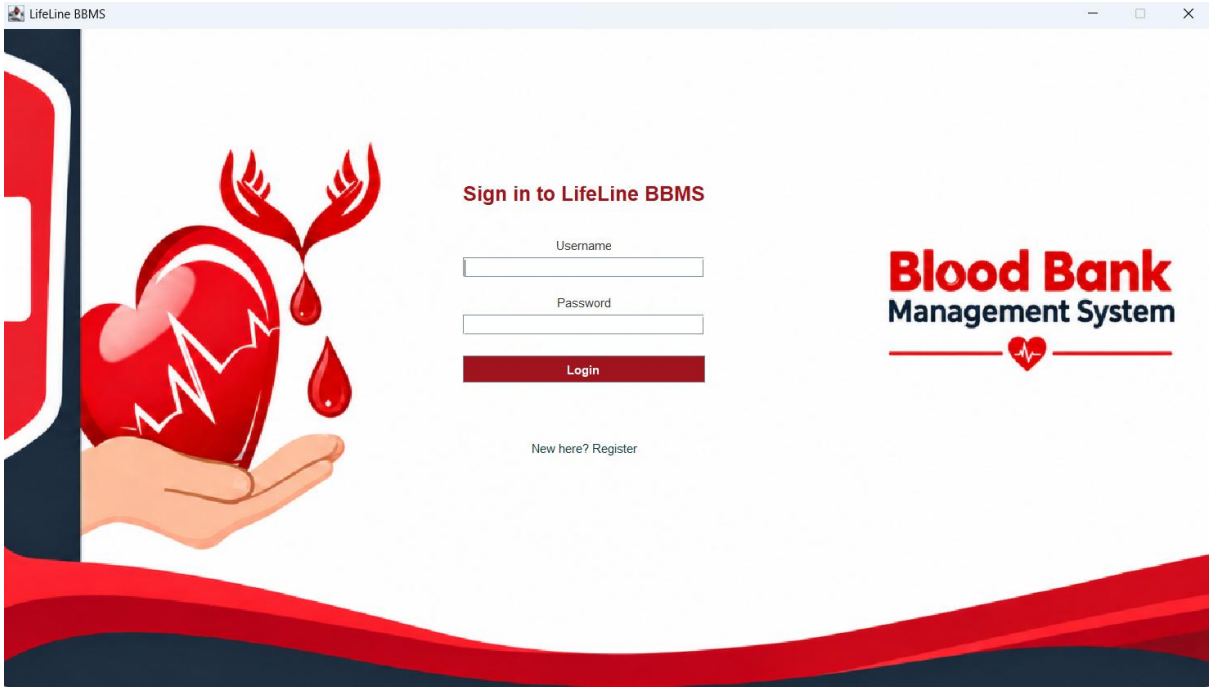


Fig: Login Screen — IntelliJ IDEA 2026.2.1

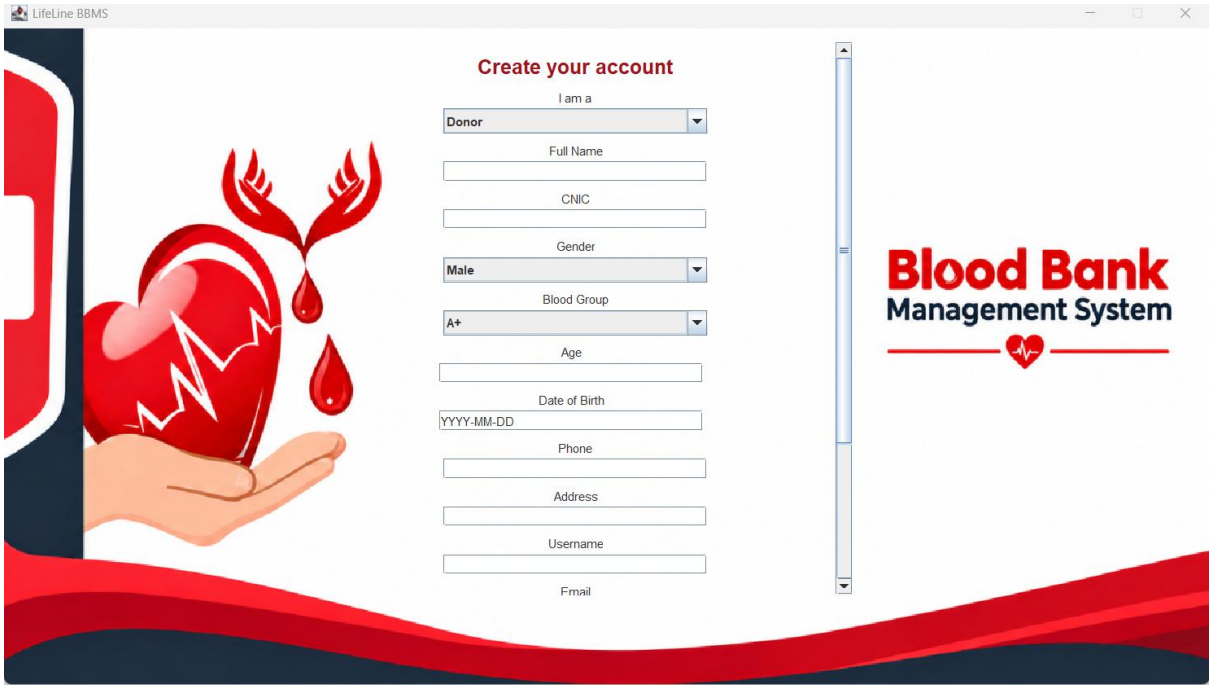


Fig: Registration Screen — IntelliJ IDEA 2026.2.1

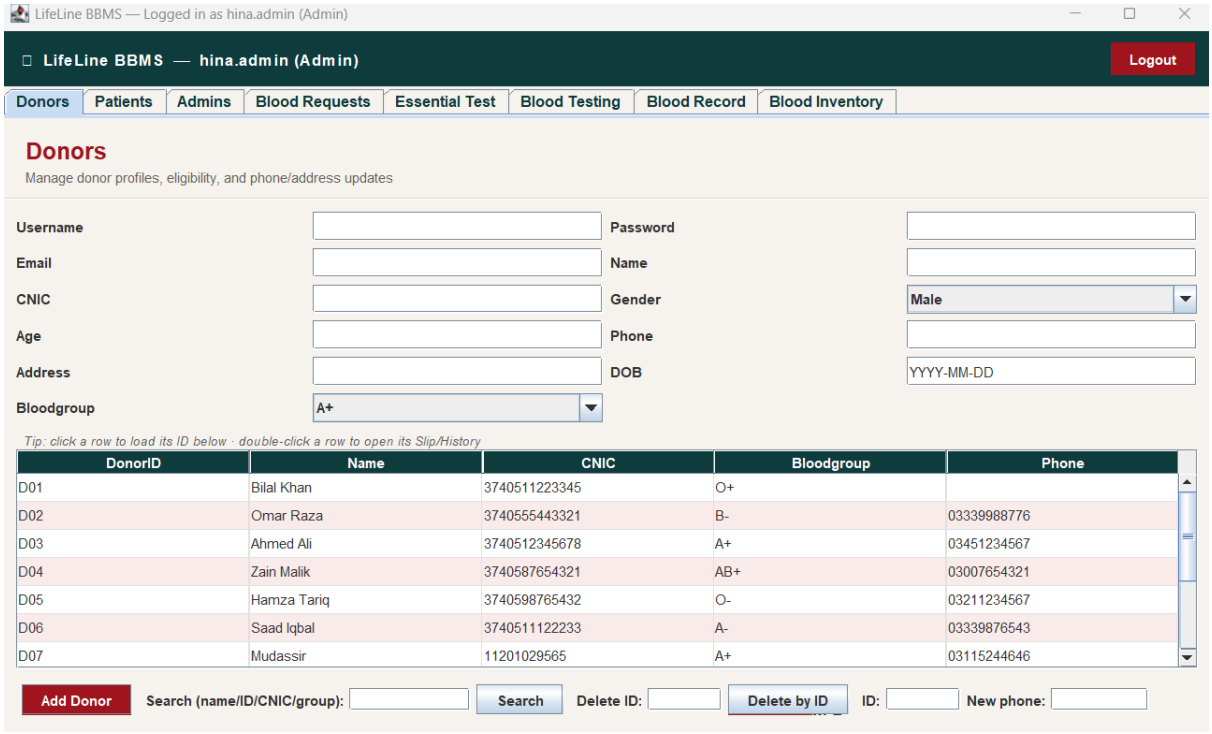
6.2 Main Dashboard (BBMSApp)

After a successful login, the main window (BBMSApp) opens with a tabbed interface. The tabs shown depend on the logged-in role — Admins see every tab, while Donors and Patients see only the tabs relevant to them:

- Donor Tab — register/view donor profiles and blood group.

- Patient Tab — register/view patient profiles.
- Admin Tab — manage admin accounts.
- Blood Request Tab — patients raise requests; admins approve/reject them.
- Screening Tab — records the Essential Test (hemoglobin, pulse, weight, blood pressure, eligibility).
- Blood Testing Tab — records HIV, Hepatitis B/C, and Syphilis test results.
- Blood Record Tab — logs each completed donation.
- Inventory Tab — tracks stock by blood group, expiry date, and availability status.

Each tab follows the same layout convention: a form (built with GridLayout) for data entry at the top, an action row (Add / Update / Delete / Refresh) below it, and a live table of records in the center. Donor and Patient tabs also support printing a summary "slip" dialog.



Main Dashboard — Admin View — IntelliJ IDEA 2026.2.1

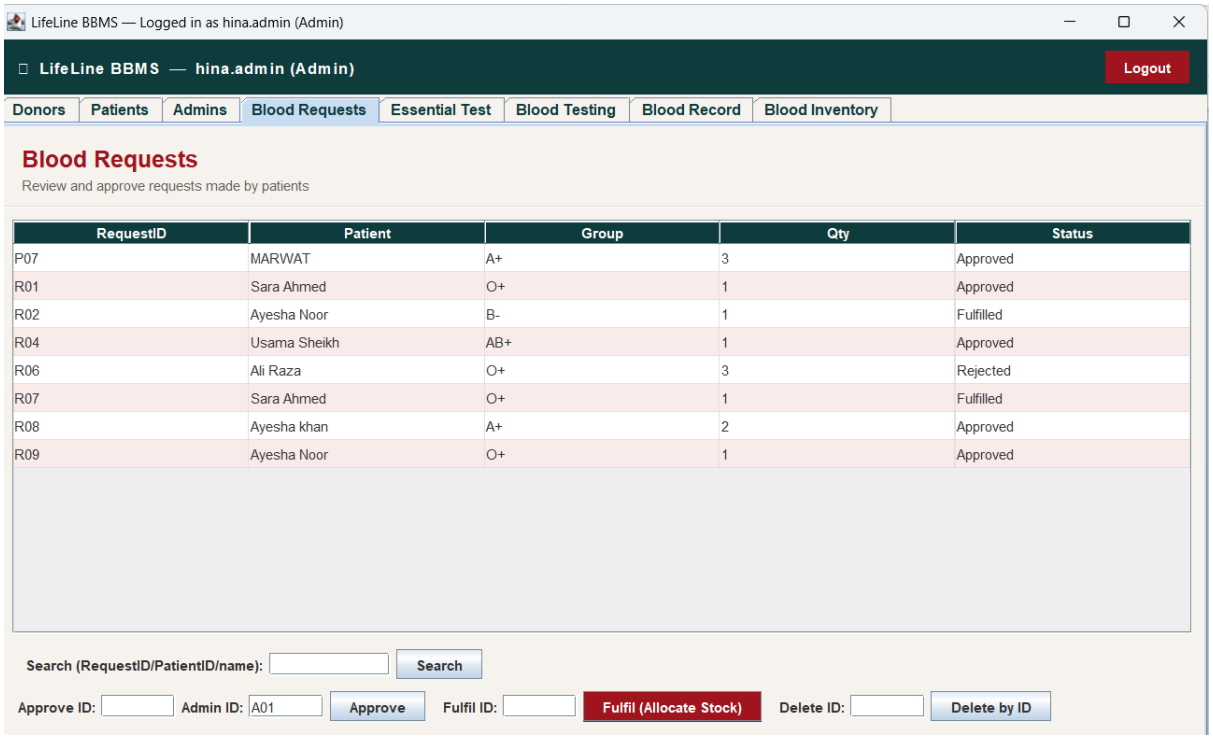
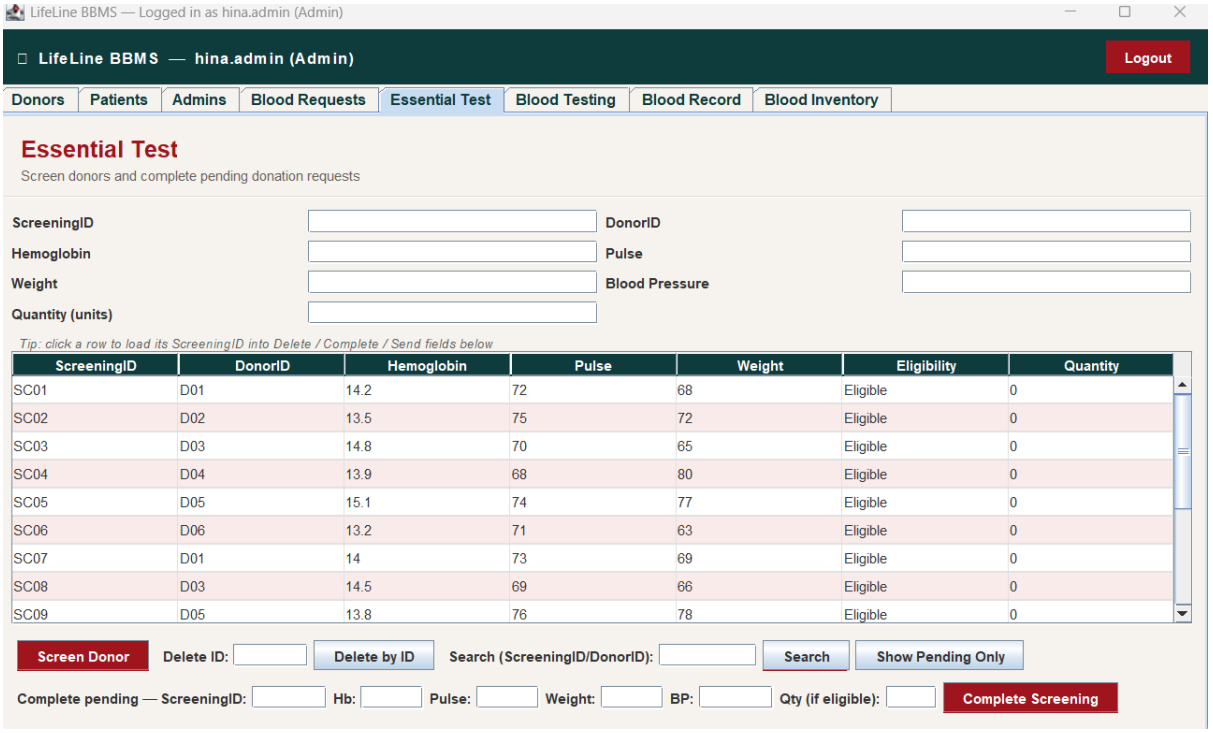


Fig: Blood Request Tab — IntelliJ IDEA 2026.2.1



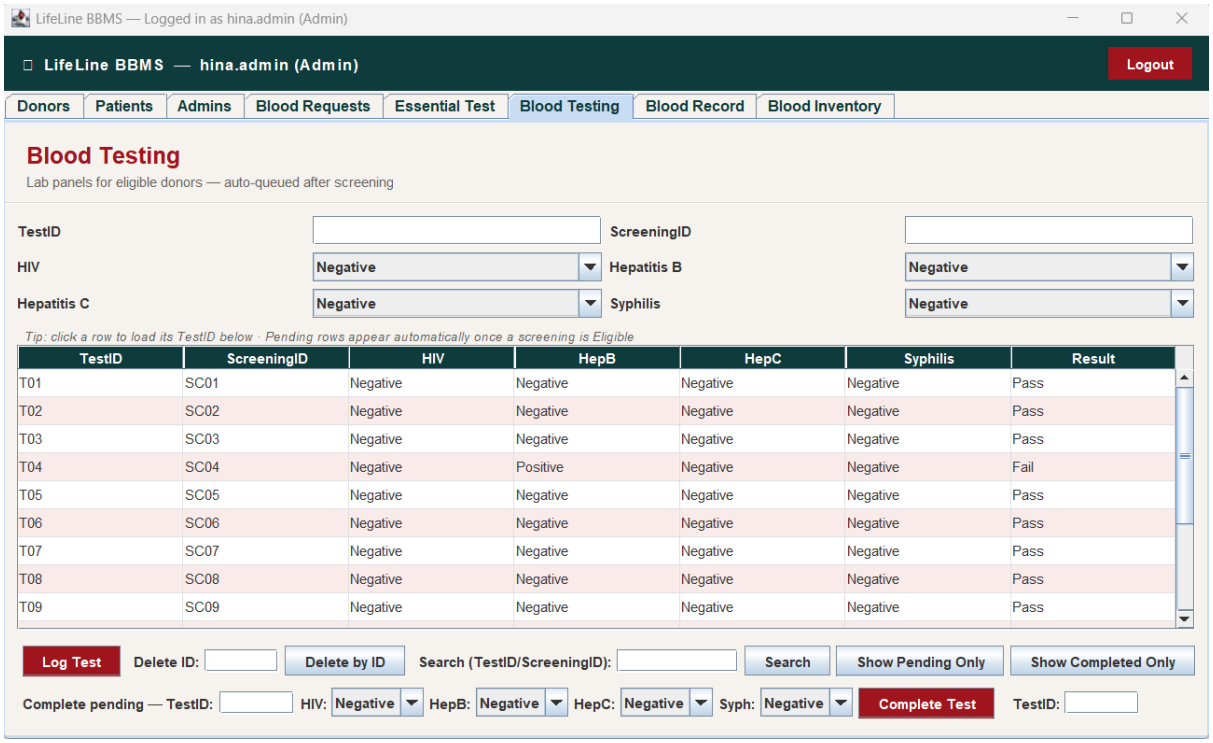


Fig: Screening / Blood Testing Tab — IntelliJ IDEA 2026.2.1

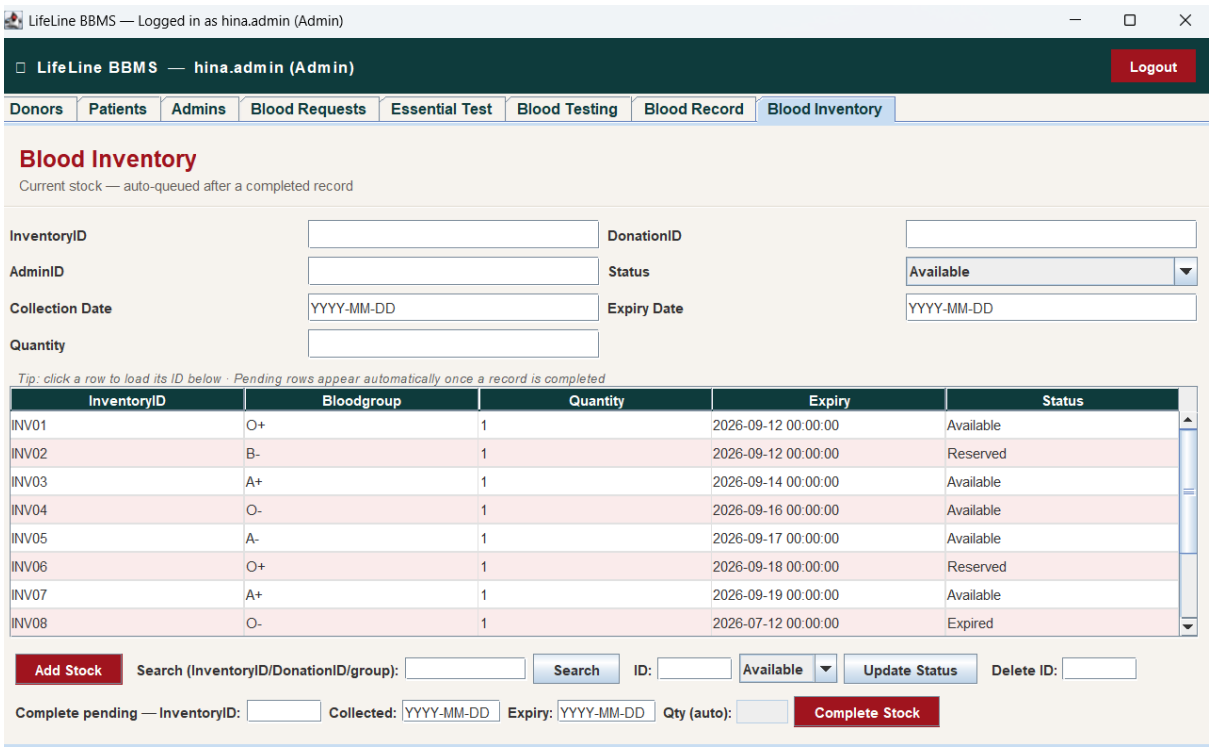


Fig: Blood Inventory Tab — IntelliJ IDEA 2026.2.1